JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent.atomic

Class AtomicIntegerFieldUpdater<T>

java.lang.Object
    java.util.concurrent.atomic.AtomicIntegerFieldUpdater<T>

Type Parameters:

`T - The type of the object holding the updatable field`

```
public abstract class AtomicIntegerFieldUpdater<T>
extends Object
```

A reflection-based utility that enables atomic updates to designated `volatile int` fields of designated classes. This class is designed for use in atomic data structures in which several fields of the same node are independently subject to atomic updates.

Note that the guarantees of the `compareAndSet` method in this class are weaker than in other atomic classes. Because this class cannot ensure that all uses of the field are appropriate for purposes of atomic access, it can guarantee atomicity only with respect to other invocations of `compareAndSet` and `set` on the same updater.

Since:

`1.5`

Constructor Summary

Constructors

Modifier	Constructor	Description

protected	**AtomicIntegerFieldUpdater**()	Protected do-nothing constructor for use by subclasses.

Method Summary

Concrete

Modifier and Type	Method	D
int	**accumulateAndGet**(**T** obj, int x, **IntBinaryOperator** accumulatorFunction)	
int	**addAndGet**(**T** obj, int delta)	
abstract boolean	**compareAndSet**(**T** obj, int expect, int update)	

int	**decrementAndGet**(**T** obj)
abstract int	**get**(**T** obj)
int	**getAndAccumulate**(**T** obj, int x, **IntBinaryOperator** accumulatorFunction)

int	**getAndAdd**(**T** obj, int delta)
int	**getAndDecrement**(**T** obj)
int	**getAndIncrement**(**T** obj)
int	**getAndSet**(**T** obj, int newValue)

int	**getAndUpdate**(**T** obj, **IntUnaryOperator** updateFunction)
int	**incrementAndGet**(**T** obj)
abstract void	**lazySet**(**T** obj, int newValue)

static <U> **AtomicIntegerFieldUpdater**<U>	**newUpdater**(**Class**<U> tclass, **String** fieldName)
abstract void	**set**(**T** obj, int newValue)
int	**updateAndGet**(**T** obj, **IntUnaryOperator** updateFunction)
abstract boolean	**weakCompareAndSet**(**T** obj, int expect, int update)

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Detail

AtomicIntegerFieldUpdater

```
protected AtomicIntegerFieldUpdater()
```

Protected do-nothing constructor for use by subclasses.

Method Detail

newUpdater

```
public static <U> AtomicIntegerFieldUpdater<U> newUpdater(Class<U> tclass,
                                                          String fieldName)
```

Creates and returns an updater for objects with the given field. The Class argument is needed to check that reflective types and generic types match.

Type Parameters:

`U` - the type of instances of tclass

Parameters:

`tclass` - the class of the objects holding the field

`fieldName` - the name of the field to be updated

Returns:

the updater

Throws:

`IllegalArgumentException` - if the field is not a volatile integer type

RuntimeException - with a nested reflection-based exception if the class does not hold field or is the wrong type, or the field is inaccessible to the caller according to Java language access control

compareAndSet

```
public abstract boolean compareAndSet(T obj,
                                      int expect,
                                      int update)
```

Atomically sets the field of the given object managed by this updater to the given updated value if the current value == the expected value. This method is guaranteed to be atomic with respect to other calls to `compareAndSet` and `set`, but not necessarily with respect to other changes in the field.

Parameters:

obj - An object whose field to conditionally set

expect - the expected value

update - the new value

Returns:

true if successful

Throws:

ClassCastException - if obj is not an instance of the class possessing the field established in the constructor

weakCompareAndSet

```
public abstract boolean weakCompareAndSet(T obj,
                                          int expect,
                                          int update)
```

Atomically sets the field of the given object managed by this updater to the given updated value if the current value == the expected value. This method is guaranteed to be atomic with respect to other calls to `compareAndSet` and `set`, but not necessarily with respect to other changes in the field.

May fail spuriously and does not provide ordering guarantees, so is only rarely an appropriate alternative to `compareAndSet`.

Parameters:

obj - An object whose field to conditionally set

expect - the expected value

update - the new value

Returns:

true if successful

Throws:

ClassCastException - if obj is not an instance of the class possessing the field established in the constructor

set

```
public abstract void set(T obj,
                         int newValue)
```

Sets the field of the given object managed by this updater to the given updated value. This operation is guaranteed to act as a volatile store with respect to subsequent invocations of compareAndSet.

Parameters:

obj - An object whose field to set

newValue - the new value

lazySet

```
public abstract void lazySet(T obj,
                             int newValue)
```

Eventually sets the field of the given object managed by this updater to the given updated value.

Parameters:

obj - An object whose field to set

newValue - the new value

Since:

1.6

get

```
public abstract int get(T obj)
```

Gets the current value held in the field of the given object managed by this updater.

Parameters:

`obj - An object whose field to get`

Returns:

`the current value`

getAndSet

```
public int getAndSet(T obj,
                     int newValue)
```

Atomically sets the field of the given object managed by this updater to the given value and returns the old value.

Parameters:

`obj - An object whose field to get and set`

`newValue - the new value`

Returns:

`the previous value`

getAndIncrement

```
public int getAndIncrement(T obj)
```

Atomically increments by one the current value of the field of the given object managed by this updater.

Parameters:

`obj - An object whose field to get and set`

Returns:

`the previous value`

getAndDecrement

```
public int getAndDecrement(T obj)
```

Atomically decrements by one the current value of the field of the given object managed by this updater.

Parameters:

`obj - An object whose field to get and set`

Returns:

`the previous value`

getAndAdd

```
public int getAndAdd(T obj,
                     int delta)
```

Atomically adds the given value to the current value of the field of the given object managed by this updater.

Parameters:

`obj - An object whose field to get and set`

`delta - the value to add`

Returns:

`the previous value`

incrementAndGet

```
public int incrementAndGet(T obj)
```

Atomically increments by one the current value of the field of the given object managed by this updater.

Parameters:

`obj - An object whose field to get and set`

Returns:

`the updated value`

decrementAndGet

```
public int decrementAndGet(T obj)
```

Atomically decrements by one the current value of the field of the given object managed by this updater.

Parameters:

`obj - An object whose field to get and set`

Returns:

the updated value

addAndGet

```
public int addAndGet(T obj,
                     int delta)
```

Atomically adds the given value to the current value of the field of the given object managed by this updater.

Parameters:

obj - An object whose field to get and set

delta - the value to add

Returns:

the updated value

getAndUpdate

```
public final int getAndUpdate(T obj,
                              IntUnaryOperator updateFunction)
```

Atomically updates the field of the given object managed by this updater with the results of applying the given function, returning the previous value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads.

Parameters:

obj - An object whose field to get and set

updateFunction - a side-effect-free function

Returns:

the previous value

Since:

1.8

updateAndGet

```
public final int updateAndGet(T obj,
                              IntUnaryOperator updateFunction)
```

Atomically updates the field of the given object managed by this updater with the results of applying the given function, returning the updated value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads.

Parameters:

`obj - An object whose field to get and set`

`updateFunction - a side-effect-free function`

Returns:

`the updated value`

Since:

`1.8`

getAndAccumulate

```
public final int getAndAccumulate(T obj,
                                  int x,
                                  IntBinaryOperator accumulatorFunction)
```

Atomically updates the field of the given object managed by this updater with the results of applying the given function to the current and given values, returning the previous value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads. The function is applied with the current value as its first argument, and the given update as the second argument.

Parameters:

`obj - An object whose field to get and set`

`x - the update value`

`accumulatorFunction - a side-effect-free function of two arguments`

Returns:

`the previous value`

Since:

`1.8`

accumulateAndGet

```
public final int accumulateAndGet(T obj,
                                  int x,
                                  IntBinaryOperator accumulatorFunction)
```

Atomically updates the field of the given object managed by this updater with the results of applying the given function to the current and given values, returning the updated value. The function should be side-effect-free, since it may be re-applied when attempted updates fail due to contention among threads. The function is applied with the current value as its first argument, and the given update as the second argument.

Parameters:

obj - An object whose field to get and set

x - the update value

accumulatorFunction - a side-effect-free function of two arguments

Returns:

the updated value

Since:

1.8

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.